

Information Systems Design

Condensed Summary of Six Lectures

This document rewrites the six lecture slide sets in the same English, but in a shorter and more readable study-note form.

Lectures covered: Origins of Software, SDLC Principles, Managing the Information Systems Project and PVF, Logical Modeling of Processes and Data Flow Diagrams, Logic Requirements, and Entity-Relationship Modeling.

Prepared from the lecture PDFs provided by the user.

Contents

1. The Origins of Software
2. The Systems Development Environment and SDLC Principles
3. Managing the Information Systems Project with the PVF Case Study
4. The Logical Modeling of Processes and Data Flow Diagrams (DFDs)
5. Logic Requirements: Structured English and Decision Tables
6. Structuring System Data Requirements with Entity-Relationship Modeling
7. Designing Databases: Slides 11 to 13

1. The Origins of Software

Learning objectives: explain outsourcing, describe six different sources of software, and explain reuse and its role in software development.

Introduction

The lecture begins from a simple point: organizations can obtain software from different sources, and they need criteria for evaluating these sources. The lecture also emphasizes that reuse has a strong impact on software development, because many systems are no longer built entirely from the beginning.

Systems Acquisition and Outsourcing

Outsourcing means turning over responsibility for some or all of an organization's information systems applications and operations to an outside firm. Instead of developing or operating every system inside the organization, a company may ask another company to perform part of the work. The slides give examples such as a company that runs payroll applications for its clients and a company that runs another organization's applications at the client site.

The reasons to outsource are practical and managerial. Outsourcing may be cost-effective, it may allow the organization to take advantage of economies of scale, and it may free internal resources for other work. It can also reduce time to market, increase process efficiencies, and support the situation in which system development is not a core activity for the organization. In other words, if building and operating software is not central to the organization's mission, outsourcing may help the organization focus on its main business.

Six Sources of Software

The lecture identifies six major sources of software for organizations. Each source gives a different balance of control, speed, cost, flexibility, and dependence on outside parties.

1. **Information technology services firms:** outside firms that provide software-related services, development, support, and sometimes full operational responsibility.
2. **Packaged software producers:** vendors who create ready-made software packages that many organizations can buy and use with some configuration.
3. **Enterprise-wide solutions:** large integrated systems that support major functions across the organization rather than solving only one isolated problem.
4. **Application service providers (ASP):** providers that host and deliver applications as a service, so the customer organization uses the software without fully managing it internally.
5. **Open source software:** software whose source code is available for use, study, modification, and distribution under the terms of its license.
6. **In-house developers:** the organization's own staff who build systems directly for internal needs.

These sources are not equally appropriate for every organization. The right choice depends on requirements, available skills, urgency, level of customization, maintenance responsibility, and long-term strategy.

Reuse in Software Development

Reuse means using previously written software resources in new applications. The lecture especially highlights objects and components as the resources most often reused. Reuse is important because it can save time, reduce repeated work, and make development more systematic.

The slides connect reuse to two development technologies:

- **Object-oriented development:** an object class encapsulates both data and behavior for common organizational entities such as Employee. Reuse in object-oriented development means using the same object classes in more than one application.
- **Component-based development:** components may be as small as objects or as large as software units that handle single business functions. Reuse here means assembling an application from many components of different size and complexity, for example a currency-conversion component or another specialized business component.

Costs and Benefits of Reuse

The lecture closes by reminding us that reuse has both costs and benefits. The benefit side includes faster development, less duplication of work, and more consistent use of tested software resources. The cost side includes the effort needed to locate reusable resources, evaluate them, understand them, adapt them, and integrate them correctly into a new application. Therefore, reuse is valuable, but it must be managed thoughtfully rather than assumed to be automatically free.

Main Idea of the Lecture

The origins of software in an organization are not limited to internal programming. Software may be acquired through outsourcing, packaged products, enterprise solutions, service providers, open source communities, or internal developers. At the same time, modern development depends strongly on reuse, especially through object-oriented and component-based development. The key managerial task is to choose the right source and the right level of reuse for the organization's goals.

2. The Systems Development Environment and SDLC Principles

Main topics: information systems analysis and design, the systems analyst, types of information systems, the Systems Development Life Cycle (SDLC), and alternative approaches to development.

Information Systems Analysis and Design

Information systems analysis and design is presented as a complex organizational process used to develop and maintain computer-based information systems. It is not the work of one programmer alone. Instead, it is performed by a team of business and systems professionals. The lecture also defines application software as computer software designed to support organizational functions or processes.

The Systems Analyst

The systems analyst is the organizational role most responsible for the analysis and design of information systems. The lecture emphasizes several analyst characteristics that are important for successful requirements determination:

- **Impertinence:** the willingness to question assumptions and not accept statements too quickly.
- **Impartiality:** the ability to examine facts and needs without personal bias.
- **Relaxing constraints:** the ability to think beyond current limits when exploring possible solutions.
- **Attention to details:** the discipline needed to capture accurate requirements and avoid hidden errors.
- **Reframing:** the ability to view a problem from another perspective and discover a better understanding.

Types of Information Systems

The lecture classifies information systems by the major roles they play in the operations and management of a business.

Operations Information Systems support day-to-day operations. They include transaction processing systems, process control systems, and enterprise collaboration systems.

Management Information Systems support managerial work. They include management information systems (MIS), decision support systems (DSS), and executive information systems (EIS).

Other Categories of Information Systems include:

- **Expert systems:** knowledge-based systems that provide expert advice and act as consultants to users, such as diagnostic or advisory systems.
- **Knowledge management systems:** systems that support the creation, organization, and dissemination of business knowledge throughout the enterprise.
- **Strategic information systems:** systems that support operations or management processes in ways that give the organization products, services, or capabilities for competitive advantage.
- **Functional business systems:** systems that support the operational and managerial applications of basic business functions such as accounting, finance, marketing, operations management, and human resource management.

Systems Development Life Cycle (SDLC)

The SDLC is the traditional methodology used to develop, maintain, and replace information systems. The lecture identifies five major phases:

1. **Planning:** an organization's total information system needs are identified, analyzed, prioritized, and arranged.
2. **Analysis:** system requirements are studied and structured.
3. **Design:** a description of the recommended solution is converted into logical and then physical system specifications.
4. **Implementation:** the information system is coded, tested, installed, and supported in the organization.
5. **Maintenance:** the information system is systematically repaired and improved over time.

Logical Design and Physical Design

The lecture gives special attention to the distinction between logical design and physical design. Logical design describes all functional features of the chosen system independently of any computer platform. Physical design transforms those logical specifications into technology-specific details from which programming and construction can be accomplished. This distinction is important because it separates what the system must do from how a particular technology will implement it.

Products of SDLC Phases

Although some slides mainly show figures, the core idea is that each SDLC phase produces specific deliverables that become inputs to the next phase. Planning leads to a clear project direction, analysis leads to structured requirements, design leads to system specifications, implementation leads to a working system, and maintenance leads to corrections and improvements. In practice, these products help teams communicate, review progress, and control quality.

Alternative Approaches to Development

The lecture explains that the SDLC can be streamlined or improved through several approaches, including prototyping, CASE tools, JAD, RAD, participatory design, Agile methodologies, the V-Model, eXtreme Programming, and object-oriented analysis and design (OOAD). These approaches do not all replace the SDLC; many of them modify how development work is performed within or across SDLC phases.

- **Prototyping** uses an early working model to clarify requirements and improve the design.
- **CASE tools** provide computer-based support for systems analysis and design activities.
- **JAD** uses structured, intensive sessions involving users, analysts, and managers to specify or review requirements.
- **RAD** aims to radically decrease design and implementation time through extensive user involvement, prototyping, JAD sessions, integrated CASE tools, and code generators.
- **Agile methodologies** support iterative, adaptive, and user-focused development.
- **V-Model** highlights the close relationship between development stages and corresponding testing stages.
- **eXtreme Programming** emphasizes short cycles, frequent feedback, and disciplined coding practices.
- **OOAD** uses object-oriented thinking to analyze and design systems.

The Traditional Waterfall SDLC

The waterfall model is introduced as the earliest SDLC approach. It is also called a linear-sequential life cycle model because one phase begins when another completes, with little backtracking and looping. The lecture identifies several problems with the waterfall approach: requirements are often “locked in” after they are determined, user involvement is limited mainly to the requirements phase, and development teams may focus too much on milestone deadlines instead of on sound development practices. These points explain why newer approaches were developed.

Main Idea of the Lecture

The lecture builds the basic environment of systems analysis and design. It explains who the systems analyst is, what types of information systems exist, how the SDLC organizes development work, and why alternative approaches such as JAD, RAD, Agile methods, and OOAD are used. Together, these ideas show that information systems development is both a technical and an organizational discipline.

3. Managing the Information Systems Project with the PVF Case Study

Main topics: Pine Valley Furniture (PVF), project management, initiating and planning systems development projects, project plans, PERT, and strategic identification and selection of projects.

Pine Valley Furniture (PVF) as the Case Study

Pine Valley Furniture is the case study used throughout the lecture. PVF manufactures high-quality wood furniture and distributes it to retail stores within the United States. The company started in the early 1980s and expanded rapidly. As the business grew, its operations became more complex and the company organized work into functional areas such as manufacturing, sales, orders, accounting, and purchasing.

PVF first relied on manual information systems, then installed a network server to automate invoicing, accounts receivable, and inventory control applications, and later upgraded to a centralized database management system. The company decided to develop new application software in-house. The new information system must take into account market conditions, the Internet, and the World Wide Web. The lecture uses this case to show why the project manager plays a key role in developing a new information system.

What Is a Project and What Is Project Management?

A project is a planned undertaking of related activities to reach an objective that has a beginning and an end. It is a unique process consisting of coordinated and controlled activities with start and finish dates and with constraints of time, cost, quality, and resources. This means a project has a defined scope and must be managed within practical limits.

Project management is described as both the art of organizing, leading, reporting, and completing a project through people and as a controlled process of initiating, planning, executing, and closing down a project. The project manager is a systems analyst with management, leadership, technical, conflict-management, and customer-relationship skills. In the PVF case, Juanita Lopez and Chris Martin are the project managers.

The Project Management Process

The lecture divides the project management process into four major stages:

1. **Initiating the project**
2. **Planning the project**

3. **Executing the project**
4. **Closing down the project**

Each stage contains activities that help the organization move from a project idea to a completed system.

Project Initiation

Project initiation is the first phase of the process. Its purpose is to assess the size, scope, and complexity of the project and to establish procedures that will support later activities. The process includes:

1. Establishing an initiation team.
2. Establishing a relationship with the customer.
3. Establishing a project initiation plan.
4. Establishing management procedures, communication procedures, and reporting procedures.
5. Establishing a project management environment and a project workbook.
6. Developing a project charter.

A **project workbook** is a repository, either online or hard copy, for all project correspondence, inputs, outputs, deliverables, procedures, and standards. It helps the team keep project information organized. A **System Service Request (SSR)** is a standard form for requesting or proposing systems development work inside an organization. It includes the contact person, problem statement, service request statement, and liaison contact information.

Project Charter

The key activity of project initiation is the development of the project charter. The project charter is a short, high-level document prepared for both internal and external stakeholders. It formally announces the project and briefly describes its objectives, key assumptions, and stakeholders. It also serves as a communication tool so that everyone understands the initiation of the project.

A project charter typically contains:

1. Project title and date of authorization
2. Project manager name and contact information
3. Customer name and contact information
4. Projected start and completion dates
5. Key stakeholders, project roles, and responsibilities
6. Project objectives and description
7. Key assumptions or approach
8. Signature section for key stakeholders

Project Planning

Project planning is the second phase of the process. It focuses on defining clear, discrete activities and the work needed to complete each activity within one project. The main objective of project planning is to develop a Baseline Project Plan (BPP) and a Project Scope Statement (PSS).

The lecture outlines ten project planning activities:

1. **Describe project scope, alternatives, and feasibility.** This step clarifies the problems or opportunities addressed by the project, the quantifiable results to be achieved, what work needs to be done, how success will be measured, and how completion will be recognized.

2. **Divide the project into manageable tasks.** This is done through a Work Breakdown Structure (WBS), which breaks the project into logically ordered tasks.
3. **Estimate resources and create a resource plan.** The lecture mentions COCOMO, a widely used method for predicting human resource requirements for systems of different complexity.
4. **Develop a preliminary schedule.** Network diagrams are used to show tasks and their interrelationships.
5. **Develop a communication plan.** This identifies stakeholders, the information each stakeholder needs, the timing and format of that information, its sources, and who is responsible for collecting, organizing, storing, verifying, and communicating it.
6. **Determine project standards and procedures.** This step specifies how deliverables will be produced and tested.
7. **Identify and assess risk.** The goal is to identify risk sources and estimate their consequences. The lecture points to factors such as project size, project structure, the development group, and the user group.
8. **Create a preliminary budget.** A preliminary budget outlines planned expenses and revenues associated with the project.
9. **Develop the Project Scope Statement (PSS).** This customer-oriented document describes the work to be done and the project deliverables at a high level. It usually contains a problem statement, project objectives, project description, business benefits, deliverables, and expected duration.
10. **Set the Baseline Project Plan (BPP).** The BPP collects and organizes all information from initiation and planning and becomes the main guide for project execution.

Baseline Project Plan (BPP)

The BPP is intended primarily to guide the development team. It contains the best estimate of a project's scope, benefits, costs, risks, and resource requirements. The lecture lists sections such as the introduction, system description, feasibility assessment, and management issues. The BPP is a major outcome of initiation and planning and becomes the main reference in later work.

Project Execution and Closedown

Project execution is the third phase, where the plans from initiation and planning are put into action. Project closedown is the final phase, which focuses on bringing the project to an end in an orderly way. Although these slides do not go deeply into execution details, they place these stages in the full management cycle.

Representing and Scheduling Project Plans

The lecture notes that project plans can be represented through graphical or textual reports, but graphical methods have become the most popular. The two most common methods are **Gantt charts** and **network diagrams**.

- **Gantt chart:** a graphical representation of a project that shows each task as a horizontal bar whose length is proportional to its completion time. It shows task durations, time overlap, and slack time in duration.
- **Network diagram:** a diagram whose arrows show activity sequence and whose nodes show activities that consume time and resources. It shows task dependencies, parallelism, and slack time in boxes.

PERT

PERT (Program Evaluation Review Technique) is presented as a method for calculating expected task durations by using optimistic, realistic, and pessimistic time estimates. The lecture gives the formula:

$$ET = (o + 4r + p) / 6$$

where ET is the expected time, o is optimistic time, r is realistic time, and p is pessimistic time. This approach allows planners to consider uncertainty instead of relying on only one time estimate.

Identifying and Selecting Systems Development Projects

The lecture then broadens the view from managing one project to identifying and selecting which IS development projects should be undertaken. Projects may be classified and ranked using value chain analysis or other evaluation criteria. Value chain analysis studies an organization's activities for making products or services to determine where value is added and where costs are incurred.

Corporate Strategic Planning

Project selection must be aligned with corporate strategic planning, which is an ongoing process that defines the mission, objectives, and strategies of an organization. The lecture highlights three components:

- **Mission statement:** makes clear what business the company is in.
- **Objective statements:** express qualitative and quantitative goals for reaching a desired future position.
- **Competitive strategy:** describes the method by which the organization attempts to achieve its mission and objectives.

The lecture lists main competitive strategy types as low-cost producer, product differentiation, and product focus. This shows that systems development projects should not be chosen in isolation; they should support the overall strategy of the organization.

Main Idea of the Lecture

Using PVF as a practical case, the lecture explains how information systems projects are initiated, planned, represented, scheduled, and selected. It connects detailed management tools such as the SSR, charter, WBS, BPP, Gantt charts, network diagrams, and PERT to the larger strategic question of why a project should be undertaken in the first place.

4. The Logical Modeling of Processes and Data Flow Diagrams (DFDs)

Learning objectives: understand logical modeling of processes, draw DFDs correctly, decompose DFDs, balance higher-level and lower-level diagrams, distinguish among DFD types, and use DFDs for analysis and electronic commerce applications.

What Process Modeling Does

The lecture introduces process modeling as a way to study how data move through a system and how work is performed on those data. The main tool used is the Data Flow Diagram (DFD). DFDs support analysis by showing how information moves between processes, data stores, and external entities.

Data Flow Diagramming Mechanics

DFDs can represent both physical and logical information systems, but they are especially useful for depicting purely logical information flows. The lecture stresses that DFDs use only four symbols and that they differ from system flowcharts, which focus on details of physical computing equipment.

Definitions and Symbols

The four basic DFD elements are:

- **Process:** work or actions performed on data inside the system.
- **Data store:** data at rest inside the system.
- **Source/Sink:** an external entity outside the system that is the origin or destination of data.
- **Data flow:** the arrows that depict the movement of data.

These simple symbols allow analysts to represent complicated systems clearly.

Context Diagram and Level-0 Diagram

A **context diagram** gives the highest-level view of the system. It shows the system as a single process and focuses on its interactions with outside entities. A **Level-0 diagram** expands that single process into the system's major processes, data flows, and data stores. Processes in a Level-0 diagram are labeled 1.0, 2.0, and so on, and may later be decomposed into lower-level DFDs.

Rules and Guidelines for DFDs

The lecture gives important DFD rules. First, the inputs to a process must be different from the outputs of that process, because the purpose of a process is to transform inputs into outputs. Second, objects on a DFD must have unique names, especially processes. These rules support clarity and prevent meaningless diagrams.

Good DFD practice also requires meaningful labels, consistency across levels, and careful attention to whether each data flow, process, and data store is really needed.

Decomposition of DFDs

Functional decomposition is an iterative process of breaking a system description down into finer detail. One process on a higher-level diagram is explained in greater detail on another diagram. Decomposition continues until no subprocess can logically be broken down any further.

The lecture identifies several levels:

- **Level-1 diagram:** results from the decomposition of a Level-0 process and labels subprocesses such as 4.1, 4.2, and so on.
- **Level-n diagram:** results from repeated nested decomposition of a process from the previous level.
- **Primitive DFD:** the lowest level of a DFD, where further decomposition is not needed.

The examples in the lecture show a Level-1 DFD for Process 4.0 and a Level-2 DFD for Process 4.3, illustrating how numbering carries the hierarchy of decomposition.

Balancing DFDs

Balancing is one of the most important ideas in DFD modeling. The lecture defines the conservation principle as conserving the inputs and outputs of a process at the next level of decomposition. A lower-level DFD is balanced with its higher-level parent process when the number and meaning of inputs and outputs are preserved.

This means that:

- The inputs to the lower-level DFD must equal the inputs to the related higher-level process.
- The outputs from the lower-level DFD must equal the outputs from the related higher-level process.

The lecture also explains **data flow splitting**. A composite data flow at a higher level may be split into different parts at the lower level, and the diagram remains balanced because the same data are involved even though they are shown in more detail.

Using DFDs as Analysis Tools

DFDs are not only drawing tools; they are analysis tools. The lecture mentions **gap analysis**, which is the process of discovering discrepancies between two or more sets of DFDs or inconsistencies within a single DFD. DFDs also help analysts identify inefficiencies in a system by making unnecessary movement, repetition, or missing processing more visible.

DFD Types

The lecture objectives specifically include the differences among current physical, current logical, new physical, and new logical DFDs. These types can be understood as follows:

- **Current physical DFD:** shows how the existing system is physically implemented now, including people, forms, and physical media.
- **Current logical DFD:** shows what the existing system does, independent of physical implementation details.
- **New logical DFD:** shows what the proposed system should do.
- **New physical DFD:** shows how the proposed system will be physically implemented.

This distinction helps analysts separate business requirements from implementation choices.

Electronic Commerce Application

The lecture connects DFDs to electronic commerce through the Pine Valley Furniture WebStore example. After a completed JAD session, analysts began translating the WebStore system structure into DFDs and identified six high-level processes. A Level-0 DFD for the WebStore is presented in the lecture, showing that DFDs are useful not only for traditional internal systems but also for web-based and e-commerce applications.

Main Idea of the Lecture

The lecture teaches how to model system processes logically through DFDs. It shows the basic symbols, the rules for drawing DFDs, the movement from context diagrams to Level-0 and lower-level diagrams, the importance of decomposition and balancing, the use of DFDs for system analysis, and the application of DFDs to an electronic commerce system.

5. Logic Requirements: Structured English and Decision Tables

Learning objectives: use Structured English, use decision tables, choose among logic-modeling techniques, and understand their application to electronic commerce.

Why Logic Modeling Is Needed

The lecture begins by explaining that data flow diagrams do not show the logic inside processes. A DFD tells us that a process exists and shows its data flows, but it does not describe the detailed decisions, steps, or sequence inside that process. Logic modeling is therefore used to represent the internal structure and functionality of processes depicted on a DFD. It can also show when processes occur.

The lecture lists several deliverables and outcomes of logic modeling, including a Structured English representation of process logic, decision tables, sequence diagrams, and activity diagrams.

Structured English

Structured English is defined as a modified form of the English language used to specify the logic of information system processes. There is no single standard form, but it typically relies on action verbs and noun phrases and avoids unnecessary descriptive language. The point is not literary style; the point is clear, disciplined, and readable process logic.

Structured English is useful when the analyst wants to describe a process in the order in which the work is performed. It is especially helpful for:

- Representing sequential steps in a process
- Showing IF-THEN-ELSE logic in a readable form
- Bridging the gap between business language and technical design

Because it stays close to ordinary English, Structured English is understandable to both technical and nontechnical readers, provided that the writing remains precise and disciplined.

Decision Tables

A decision table is a matrix representation of decision logic. It specifies the possible conditions for a decision and the resulting actions. The lecture notes that decision tables are best used for complicated decision logic, especially when many conditions interact and many possible rules must be checked.

The core terms of a decision table are:

- **Condition stubs:** the part of the table that lists the conditions relevant to the decision.
- **Action stubs:** the part that lists the actions that may result.
- **Rules:** the columns or combinations that specify which actions are taken for a given set of conditions.
- **Indifferent condition:** a condition whose value does not affect which action is taken for two or more rules.

Procedure for Creating Decision Tables

The lecture gives a direct procedure for creating decision tables:

1. Name the conditions and the values each condition can assume.
2. Name all possible actions that can occur.
3. List all possible rules.
4. Define the actions for each rule.
5. Simplify the table.

This procedure helps analysts build complete and consistent logic descriptions, particularly when verbal explanations become too confusing.

Choosing Between Structured English and Decision Tables

The lecture includes a comparison of the two techniques. The basic message is that each technique is stronger for a different purpose.

Criteria	Structured English	Decision Tables
Determining conditions and actions	Second best	Third best
Transforming conditions and actions into sequence	Best	Third best
Checking consistency and completeness	Third best	Best

From this comparison, we can say that Structured English is better when the sequence of steps is the main concern, while decision tables are better when the analyst must test completeness and consistency in complicated decision logic.

Electronic Commerce Application

The lecture applies logic modeling to the Pine Valley Furniture WebStore. The development methodology for PVF required that the logic within each unique process be represented. Structured English was chosen for modeling that logic. The lecture also links this work to a Level-0 DFD for the WebStore system, showing that logic modeling is connected to earlier process-modeling work and continues the detailed analysis of system behavior.

How Logic Modeling Supports Analysis

Logic modeling helps analysts move from a high-level process name to an exact statement of what that process really does. Once a DFD identifies a process, logic modeling gives the internal detail. This is especially important in systems where many business rules determine which actions are permitted, required, delayed, or rejected.

Main Idea of the Lecture

The lecture explains that analysts need tools beyond DFDs to represent the logic inside processes. Structured English provides a readable way to describe step-by-step process logic, while decision tables provide a powerful way to represent complex conditional logic. Both are important in the development of information systems, including e-commerce applications such as the PVF WebStore.

6. Structuring System Data Requirements with Entity-Relationship Modeling

Main topics: conceptual data modeling, entities, attributes, keys, relationships, degree, cardinality, ordinality, crow's foot notation, and drawing E-R diagrams.

Why Data Modeling Is Needed

The lecture starts by contrasting data-flow diagrams with data modeling. DFDs show how, where, and when data are used or changed in an information system, but they do not show the definition, structure, and relationships within the data. Data modeling provides this missing piece. It focuses on the data themselves rather than on the movement of data through processes.

Conceptual Data Modeling

A conceptual data model is a detailed model that shows the overall structure of organizational data while remaining independent of any database management system or other implementation considerations. The lecture emphasizes that conceptual data modeling is suitable during the planning and analysis phases of the SDLC, where the concern is system scope, general requirements, and content rather than implementation technology.

Most organizations perform conceptual data modeling through entity-relationship modeling. As a result, the primary deliverable of conceptual data modeling is an **Entity-Relationship (E-R) diagram**. During design, the conceptual data model is translated into a physical design. The repository or project dictionary links data modeling with the other SDLC models, so data elements in a DFD must appear in the data model and each data store in the process model must correspond to business objects in the data model.

The Conceptual Data Modeling Process

1. Develop a data model for the current system.
2. Develop a new conceptual data model that includes all requirements of the new system.
3. Translate the conceptual data model into physical design during the design stage.
4. Link all design and data-modeling steps in the project repository throughout the SDLC.

The lecture also notes that several E-R diagrams may be produced and analyzed, including diagrams for the project application, the application being replaced, and the whole database contexts from which data are drawn.

Introduction to the E-R Model

An E-R model is a detailed, logical representation of the entities, associations, and data elements for an organization or business area. An E-R diagram is the graphical representation of that model. The model is expressed in terms of data entities, relationships among those entities, and attributes of both entities and relationships.

Entity, Entity Type, and Entity Instance

An **entity** is a person, place, object, event, or concept in the user environment about which data are maintained. Examples include EMPLOYEE, STUDENT, PATIENT, STATE, COUNTRY, MACHINE, PRODUCT, SALE, ACCOUNT, or COURSE. In database terms, the lecture often treats an entity as a table.

An **entity type** is a collection of entities that share common properties or characteristics. Its name should be a singular noun, specific to the organization, and concise. In E-R notation the entity type name is written in a rectangle.

An **entity instance** is a single occurrence of an entity type. An entity type is defined once in the model, but many instances of that type may exist in the database. The lecture warns students not to confuse true data entities with system users, outputs, or external sources/sinks from DFDs. A real entity should have many possible instances and descriptive data about those instances.

Strong and Weak Entities

A **strong entity** has a primary key and can be uniquely identified by its own attributes. A **weak entity** cannot be uniquely identified by its own attributes alone. It is existence-dependent on a parent entity and usually combines its own attributes with a foreign key from the parent to form a primary key. The lecture explains that the relationship between strong and weak entities is a strong relationship because the weak entity depends on the strong one.

Attributes

An **attribute** is a property or characteristic of an entity that is of interest to the organization. Attributes provide additional information about an entity. Examples include Student_ID, Student_Name, Address, Phone_Number, Vehicle_ID, Color, Employee_ID, and Skill. Attributes are usually named as nouns and should be unique. Relationships may also have attributes.

Types of Attributes

The lecture describes several important attribute types:

- **Simple attribute:** an atomic attribute that cannot be divided further.
- **Composite attribute:** an attribute that can be broken into meaningful component parts, such as Name into first, middle, and last name.
- **Single-valued attribute:** an attribute that stores only one value for each entity instance.
- **Multivalued attribute:** an attribute that may take more than one value for each entity instance, such as multiple email addresses or contact numbers.
- **Derived attribute:** an attribute whose value can be computed from related attributes, such as Age derived from Date of Birth.
- **Repeating group attribute:** a logically related set of two or more multivalued attributes.
- **Required attribute:** an attribute that must have a value for every entity instance.
- **Optional attribute:** an attribute that may not have a value for every entity instance.

Candidate Keys, Primary Keys, Identifiers, and Foreign Keys

The lecture objectives mention candidate keys and identifiers, and the slides explicitly explain primary and foreign keys. A **candidate key** is an attribute, or a combination of attributes, that can uniquely identify an entity instance. One candidate key is chosen as the **primary key (PK)**, which must contain unique values and cannot be null. The primary key serves as the main identifier of entity instances.

A **foreign key (FK)** is a column or group of columns in one table that references a primary key in another table, thereby linking related data. Foreign keys do not have to be unique and an entity may contain several foreign keys. Through PKs and FKs, E-R relationships can later be implemented in relational database tables.

Relationships and Their Degree

A **relationship** is an association between instances of one or more entity types that is of interest to the organization. Relationships are labeled with verb phrases because they express how entity instances are associated. For example, EMPLOYEE completes COURSE.

The **degree** of a relationship is the number of entity types that participate in it:

- **Unary relationship:** a relationship among instances of one entity type, also called a recursive relationship.
- **Binary relationship:** a relationship between instances of two entity types. This is the most common case.

- **Ternary relationship:** a simultaneous relationship among instances of three entity types.

Cardinality and Ordinality

Cardinality specifies how many instances of one entity can or must be associated with each instance of another entity. The lecture distinguishes:

- **Minimum cardinality:** the minimum number of instances that may be associated.
- **Maximum cardinality:** the maximum number of instances that may be associated.

Ordinality is closely linked to cardinality. It shows whether the participation of an instance in a relationship is mandatory or optional. In other words, ordinality tells us whether an instance must exist in the relationship or may be absent.

Crow's Foot Notation

The lecture asks students to use crow's foot notation. In this notation, a pair of icons appears at each end of a relationship line. The outer icon represents the maximum value and the inner icon represents the minimum value. This notation clearly shows one-to-one, one-to-many, and many-to-many relationships, as well as optional and mandatory participation. The lecture's garden and plant-species example illustrates how a specific garden may have zero or more plant species, while each plant species must be associated with at least one garden instance.

Tips for Designing the Diagram

1. Identify all needed entities and draw them in the diagram.
2. Determine which entities have relationships and connect them appropriately.
3. Ensure that each entity appears only once in the diagram.
4. Review the relationships to find redundancy, unnecessary links, or missing links.

The lecture also contains example diagrams, Microsoft Visio equivalents, and shape references to help students see how E-R ideas are represented visually in practice.

Main Idea of the Lecture

The lecture explains how to structure system data requirements by building a conceptual data model and expressing it with an E-R diagram. It covers the core vocabulary of entities, entity types, instances, attributes, keys, relationships, degree, cardinality, ordinality, and crow's foot notation. Together, these concepts allow analysts to describe the structure of organizational data in a precise form that later supports database design.

7. Designing Databases: Slides 11 to 13

Covered only from this lecture: transforming E-R diagrams into relations, representing entities, and representing relationships.

Transforming E-R Diagrams into Relations

The lecture explains that it is useful to transform the conceptual data model into a set of normalized relations. This means that the E-R diagram is moved into relational form so that it can later be implemented in a database design. The slides summarize this transformation in four steps:

1. Represent entities.

2. Represent relationships.
3. Normalize the relations.
4. Merge the relations.

This sequence shows that the E-R model is not the final design by itself. It is a conceptual starting point that is translated into relational tables and then refined.

1. Representing Entities

Each regular entity is transformed into a relation. The identifier of the entity type becomes the primary key of the corresponding relation. The primary key must satisfy two conditions. First, the value of the key must uniquely identify every row in the relation. Second, the key should be nonredundant. The slides also note that the entity type label is translated into a relation name. In practical terms, this means that each major business entity in the E-R model becomes a table, and its identifier becomes the main key of that table.

2. Representing Relationships

The procedure for representing relationships depends on both the degree of the relationship, such as unary, binary, or ternary, and the cardinalities of the relationship.

For a **binary 1:N relationship**, the primary key attribute, or attributes, of the entity on the one side of the relationship is added as a foreign key in the relation that is on the many side of the relationship.

For a **binary or unary 1:1 relationship**, the slides list several possible choices: add the primary key of A as a foreign key of B, add the primary key of B as a foreign key of A, or use both of these choices if needed by the design.

For **binary and higher-degree M:N relationships**, the relationship is represented by creating another relation and including the primary keys of all participating relations as the primary key of the new relation. This new relation acts as the bridge that connects the related entities.

Main Idea of These Slides

Slides 11 to 13 explain the first practical step of moving from E-R modeling to relational database design. Entities become relations, identifiers become primary keys, and relationships are represented differently depending on whether they are 1:N, 1:1, or M:N. These rules are the foundation for converting a conceptual model into a workable database structure.